

A PROJECT ON
“IndiaMart – Multivendor E-commerce Marketplace”

SUBMITTED IN
PARTIAL FULFILLMENT OF THE REQUIREMENT
FOR THE COURSE OF
POST GRADUATION CERTIFICATE PROGRAMME IN ADVANCED COMPUTING
FROM CDAC



SUNBEAM INSTITUTE OF INFORMATION TECHNOLOGY
Hinjawadi

SUBMITTED BY:

Mr. SAZID MUGAL
Mr. SUMAN MONDAL
Mr. PRATHAMESH RAYKE
Mr. RISHU MADDHESHIYA

UNDER THE GUIDENCE OF:

Mr. SNEHAL JADHAV
Faculty Member
Sunbeam Institute of Information Technology, PUNE.



CERTIFICATE

This is to certify that the project work under the title 'IndiaMart – Multivendor E-commerce Platform' is done by Suman Mondal in partial fulfillment of the requirement for award of Post Graduation Certificate Programme in Advanced Computing Course.

Snehal Jadhav
Project Guide

Mr. Yogesh Kolhe
Course Co-Coordinator

Date: 10/08/2026

ACKNOWLEDGEMENT

A project usually falls short of its expectation unless aided and guided by the right persons at the right time. We avail this opportunity to express our deep sense of gratitude towards Mr. Nitin Kudale (Center Coordinator, SIIT, Pune) and Mr. Yogesh Kolhe (Course Coordinator, SIIT, Pune).

We are deeply indebted and grateful to them for their guidance, encouragement and deep concern for our project. Without their critical evaluation and suggestions at every stage of the project, this project could never have reached its present form.

Last but not the least we thank the entire faculty and the staff members of Sunbeam Institute of Information Technology, Pune for their support.

Mr. Suman Mondal

Mr. Prathamesh Rayke

Mr. Sazid Mugal

Mr. Rishu Maddheshiya

PGCP-AC

SIIT Pune

1. INTRODUCTION TO PROJECT

The Multi-Vendor E-Commerce Marketplace is a web-based distributed application designed to connect customers, vendors and administrators through a secure and scalable online commerce platform. The project follows a microservices architecture in which business capabilities are separated into independently deployable services. The platform supports user and vendor onboarding, authentication, centralized authorization, product management, order processing, payment processing and supporting services.

The system is designed around a central API Gateway. Clients communicate with the gateway rather than directly accessing individual backend services. Authentication-related operations are handled by the User Service, while authorization is centralized at the API Gateway. After validating the JSON Web Token and extracting the user identity and role, the gateway applies role-based access rules and forwards trusted user information to downstream services.

The User Service provides customer and vendor registration, OTP verification, login, token refresh, password update and password recovery functionality. During registration, OTP data and pending signup information are stored temporarily in Redis and an email notification is produced. Passwords are protected using BCrypt hashing as part of the authentication process.

The Product Service provides product creation, retrieval, update, deletion, publishing and searching functionality. It also supports category-based retrieval, product ratings, best-selling products and generation of pre-signed URLs for image uploads to Amazon S3. Vendor identity and role are received from the gateway through trusted request headers.

The project is implemented as a set of twelve microservices along with infrastructure components. The report documents the architecture and the implementation details supported by the source files supplied for the project, with particular emphasis on User Service, API Gateway authorization, Product Service, Order Service, Payment Service and administrative operations.

2. REQUIREMENTS

2.1 FUNCTIONAL REQUIREMENTS

User Registration: Customers and vendors can initiate registration by submitting their details. An OTP is generated and the signup data is temporarily stored in Redis until verification.

OTP Verification: The submitted OTP is verified before customer or vendor registration is completed. Invalid or expired OTPs are rejected.

Authentication: Registered users can log in and receive authentication tokens. Refresh-token processing is provided through the authentication service.

Password Management: Authenticated users can update passwords. Forgotten passwords can be reset through an OTP-based recovery flow.

Centralized Authorization: The API Gateway validates JWTs and applies role-based authorization rules for administrator, vendor and customer resources.

Product Management: Authorized vendors can add, edit, delete and publish/unpublish products. Product retrieval, category filtering and product search are also supported.

Product Rating and Analytics: Customers can submit or update product ratings, while best-selling product information can be retrieved.

Image Upload Support: The Product Service generates pre-signed URLs for secure image upload to Amazon S3 without routing large image files through the application server.

Order Management: Customers can place and view orders. Vendors can view their orders, inspect vendor-order details and update order status. Vendor and administrator analytics endpoints are also provided.

Payment Processing: The Payment Service processes payment webhooks, retrieves recent payments and provides payment details using the payment provider identifier.

Administration: Administrative operations include viewing customers and vendors, updating vendor details, deleting vendors and changing user ban/unban status.

Notification Management: The Notification Service consumes notification events through RabbitMQ and sends email notifications to users for relevant events such as registration, order updates and payment status changes.

Centralized Logging: The Logging Service collects and stores application logs from different microservices, providing centralized monitoring and easier tracking and debugging of errors and system activities.

2.2 NON-FUNCTIONAL REQUIREMENTS

- Security: JWT-based authentication with centralized role-based authorization at the API Gateway and BCrypt password hashing.
- Scalability: Independent microservices can be scaled according to their individual load.
- Maintainability: Business responsibilities are separated into services and layered into controllers, services and persistence components.
- Availability: Failure of a non-critical service should be isolated as far as possible from unrelated business capabilities.
- Performance: Redis is used for temporary OTP/signup data and the gateway avoids unnecessary authorization logic inside every downstream controller.
- Extensibility: Additional microservices and endpoints can be introduced without redesigning the entire application.

3. SYSTEM ARCHITECTURE AND DESIGN

3.1 MICROSERVICE ARCHITECTURE

The application follows a microservice-based architecture. Each business capability is implemented as an independently deployable service. The client communicates with the API Gateway, which performs request routing and centralized

authorization. The User Service handles identity-related operations, while Product, Order and Payment services handle their respective business domains.

Layer	Component	Responsibility
Client	Web / React Frontend	Provides customer, vendor and administrator user interfaces.
Edge	API Gateway	Routes requests, validates JWTs, applies centralized authorization and forwards user identity headers.
Identity	User Service	Registration, OTP verification, login, token refresh and password management.
Business	Product Service	Product CRUD, search, ratings, publishing, image upload support and best-selling products.
Business	Order Service	Order creation, customer/vendor order views, status updates and analytics.
Business	Payment Service	Payment webhook handling and payment information retrieval.
Infrastructure	Redis	Temporary OTP and signup-data storage.
Infrastructure	Message/Notification component	Publishes email notification requests for OTP and related communication.
Infrastructure	Object Storage	Amazon S3 for product image storage through pre-signed upload URLs.

3.2 TWELVE-MICROSERVICE DEPLOYMENT MODEL

The complete project contains twelve microservices. The exact service boundaries and responsibilities of every service should be finalized from the remaining service source files before the final institutional submission. The supplied implementation confirms User/Authentication, Product, Order, Payment and administrative functionality, together with API Gateway authorization and supporting infrastructure. This report therefore avoids inventing undocumented business behavior for the remaining services.

3.3 REQUEST FLOW

A protected request begins at the frontend and reaches the API Gateway with a Bearer token. The gateway validates the token, extracts user ID, email and role, checks the requested path and HTTP method against authorization rules, and rejects unauthorized requests. For authorized requests, the gateway forwards the request with X-User-Id, X-User-Email and X-User-Role headers. Downstream services can then use these trusted values for business-level ownership checks.

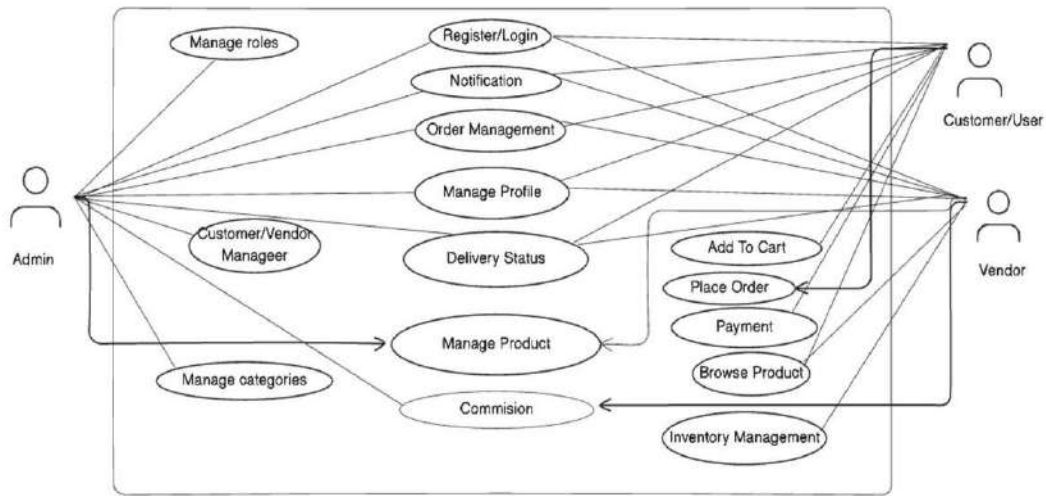
Step	Action
1	Client sends HTTP request to API Gateway.
2	Gateway identifies whether the endpoint is public or protected.
3	For protected endpoints, the Bearer JWT is extracted and validated.
4	User ID, email and role are extracted from the token.
5	Gateway checks role against the requested path and HTTP method.
6	If authorized, trusted user headers are attached to the forwarded request.
7	The target microservice executes business logic and returns a response.
8	Gateway forwards the response to the client.

4. TECHNOLOGY STACK

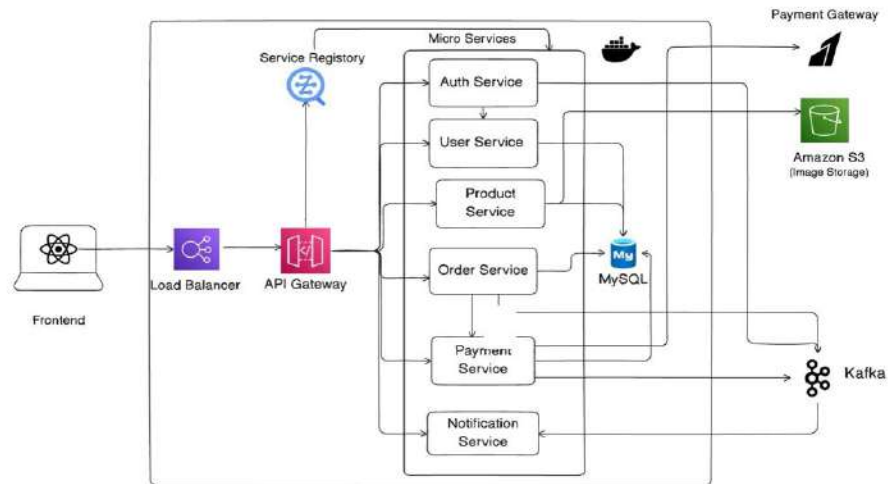
Technology	Usage
Java	Backend programming language.
Spring Boot	Application framework used to implement microservices and REST APIs.
Spring Cloud Gateway	API Gateway and centralized request filtering/authorization.
Spring Security concepts / JWT	Token validation and security flow.
BCrypt	Secure one-way password hashing.
REST / HTTP	Communication interface between frontend, gateway and services.

MySQL / relational database	Persistent storage for business data where applicable.
Redis	Temporary OTP and registration-data storage.
RabbitMQ / messaging	Asynchronous notification/event communication where configured.
Amazon S3	Product image object storage using pre-signed URLs.
Cashfree	Payment processing/webhook integration.
Maven	Dependency management and build automation.
Docker	Containerization and service deployment support.
React	Frontend client technology.

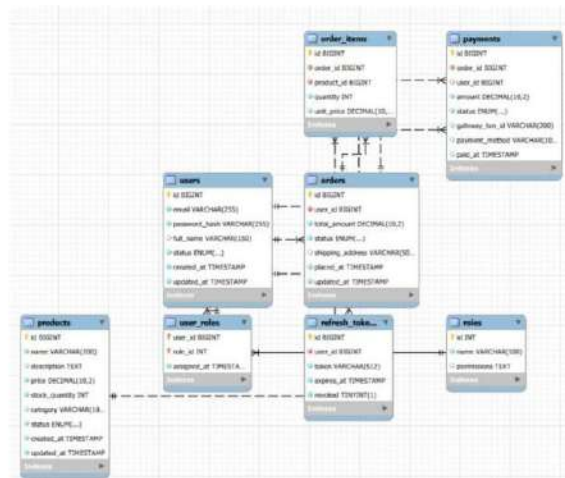
4.5 USE CASE DIAGRAM



4.6 SYSTEM DESIGN



4.7 ENTITY RELATIONSHIP DIAGRAM



5. DETAILED MODULE DESIGN

5.1 USER SERVICE – REGISTRATION AND AUTHENTICATION

The User Service is responsible for identity and account-related operations. The authentication controller exposes customer signup, vendor signup, OTP verification, login, refresh-token, password update and password recovery endpoints. Customer and vendor signup flows generate an OTP, temporarily store the submitted signup information in Redis and publish an email notification. The account is created only after successful OTP verification.

The authentication flow uses BCrypt to protect passwords rather than storing plaintext passwords. During login, the submitted credentials are checked against the stored password hash. On successful authentication, the service returns token information used by the API Gateway for subsequent requests.

Endpoint	Method	Purpose
/auth/customers/signup	POST	Initiates customer signup and sends OTP.
/auth/vendor-signup	POST	Initiates vendor signup and sends OTP.
/auth/verify-vendor-signup	POST	Verifies vendor OTP and completes registration.
/auth/customers/verify-signup	POST	Verifies customer OTP and completes registration.
/auth/login	POST	Authenticates a registered user.
/auth/refresh	POST	Generates/returns a refreshed access token using refresh token.
/auth/update-password	PATCH	Updates password for an authenticated user.
/auth/forget-password	POST	Initiates forgotten-password OTP flow.
/auth/reset-password	POST	Resets password after recovery verification.

The supplied authentication controller demonstrates that OTP generation and temporary signup data are handled through an OTP service backed by Redis, while email

notifications are produced asynchronously. This reduces the need to persist incomplete registration records permanently and improves separation between registration and notification concerns.

5.2 API GATEWAY – CENTRALIZED AUTHORIZATION

The API Gateway contains a GlobalFilter-based JWT authorization component. The filter first permits configured public endpoints, while protected endpoints require an Authorization header containing a Bearer token. The JWT is validated and user ID, email and role are extracted. Authorization is then performed centrally using the requested URL path and HTTP method.

Role-based rules are applied for administrator, vendor and customer operations. For example, adding a product, viewing a vendor's own products and editing products are restricted to vendors; deleting products is restricted to vendors or administrators; product ratings are restricted to customers; customer order operations are restricted to customers; vendor order operations are restricted to vendors; and administrator analytics are restricted to administrators.

Role	Examples of authorized operations
CUSTOMER	Create orders, view own orders, rate products, access permitted customer resources.
VENDOR	Add/edit/delete permitted products, view own products, publish/unpublish products, view vendor orders and analytics.
ADMIN	Access administrative resources, manage vendors, view payment information, access administrative order details and analytics.

After authorization, the gateway forwards X-User-Id, X-User-Email and X-User-Role headers to downstream services. This allows services to perform ownership checks without repeatedly decoding the JWT in every controller.

5.3 PRODUCT SERVICE

The Product Service provides the main product-management functionality of the marketplace. The controller exposes product creation, product retrieval, category filtering, vendor-specific product listing, update, delete, publish/unpublish, product count, search, image-upload URL generation, ratings and best-selling product retrieval.

Endpoint	Method	Access / Purpose
/api/products	POST	Vendor – add a product.
/api/products/{productId}	GET	Public – retrieve product details.
/api/products	GET	Public – retrieve products by category.
/api/products/myproducts	GET	Vendor – retrieve own products.
/api/products/Edit	PUT	Vendor – update product.
/api/products/{productId}	DELETE	Vendor/Admin – delete product.
/api/products/{productId}/publish-unpublish	PATCH	Vendor – toggle publish status.
/api/products/product-count	GET	Vendor – retrieve own product count.
/api/products/search	GET	Public – search products by name.
/api/products/presigned-url	GET	Vendor – obtain S3 upload URL.
/api/products/{productId}/rating	PATCH	Customer – add/update rating.
/api/products/best-selling	GET	Public – retrieve best-selling products.

The ProductController receives vendorId and userRole from the gateway for protected operations. Product images are handled through an AwsS3Service that generates a pre-signed URL, allowing the client to upload an object directly to S3. Product ratings are delegated to a ProductRatingService, while best-selling information is delegated to ProductSalesService.

5.4 ORDER SERVICE

The Order Service supports order creation and retrieval for customers and vendors. The controller exposes customer order listing, individual order retrieval, vendor order listing, vendor-order detail retrieval, vendor order status updates and dashboard analytics.

Endpoint	Method	Purpose
/api/orders	POST	Create a new customer order.
/api/orders/customer	GET	Retrieve all orders for a customer.
/api/orders/{orderId}	GET	Retrieve order details.
/api/orders/vendor	GET	Retrieve all orders for a vendor.
/api/orders/vendor/{vendorOrderId}	GET	Retrieve vendor-order details.
/api/orders/vendor/update-status/{vendorOrderId}	PATCH	Vendor updates vendor-order status.
/api/orders/vendor/analytics	GET	Vendor dashboard analytics.
/api/orders/admin/analytics	GET	Administrator dashboard analytics.

The Order Service obtains customerId or vendorId from X-User-Id headers supplied by the API Gateway. Validation annotations are used for important identifiers such as customer ID, vendor ID and order IDs.

5.5 PAYMENT SERVICE

The Payment Service integrates payment processing with a payment provider and exposes a webhook endpoint for payment-status processing. The webhook accepts the raw request body together with the signature and timestamp headers so that the service can verify the callback before updating payment status. The service also provides recent payment retrieval and payment-detail lookup.

Endpoint	Method	Purpose
/api/payments/webhook	POST	Process and verify payment provider webhook.
/api/payments/recent	GET	Retrieve recent payments.
/api/payments/{cfPaymentId}	GET	Retrieve payment details by provider payment ID.

5.6 ADMINISTRATIVE MODULE

Administrative operations are exposed through the AdminController. The administrator can retrieve customer details, retrieve all customers, update customer profile information, ban or unban a user, retrieve all vendors, update vendor details and

delete vendors. Administrative access is protected at the gateway by the administrator role.

Endpoint	Purpose
/admin/customers/{id}	Retrieve customer details.
/admin/customers/all	Retrieve all customer details.
/admin/customers/profile	Update customer profile.
/admin/ban-unban	Change user ban/unban status.
/admin/vendors/all	Retrieve all vendors.
/admin/update-vendor/{id}	Update vendor information.
/admin/delete-vendor/{id}	Delete a vendor.

6. DATABASE AND DATA MANAGEMENT

Each business service is responsible for its own domain data and persistence concerns. This separation reduces coupling between services and supports independent evolution. Entity and repository details should be finalized from the complete database/entity files supplied by the team.

Domain	Typical Data Managed
User / Identity	Customer accounts, vendor accounts, credentials, roles and account status.
Product	Products, category, vendor ownership, publication status, ratings and sales-related information.
Order	Orders, vendor-order details, order status and customer/vendor relationships.
Payment	Payment identifiers, payment status and payment-provider callback information.
Temporary Registration	OTP and incomplete signup information stored temporarily in Redis.
Object Storage	Product image objects stored in Amazon S3.

6.1 REDIS OTP FLOW

Stage	Operation
1	User submits signup details.
2	OTP service generates a one-time password.

3	Signup DTO and OTP are stored temporarily in Redis.
4	Notification producer sends an email containing the OTP.
5	User submits OTP for verification.
6	OTP service verifies the code and expiration.
7	On success, stored signup data is retrieved and permanent registration is completed.

7. SECURITY DESIGN

7.1 AUTHENTICATION

Authentication verifies the identity of the user. The User Service accepts login credentials and returns token information after successful credential verification. Passwords are stored using BCrypt hashing so that the original password is not persisted in plaintext. Access and refresh token handling allows the client to continue using the application without repeatedly submitting credentials.

7.2 AUTHORIZATION

Authorization determines whether an authenticated user is allowed to perform an operation. In this project, authorization is centralized in the API Gateway. The gateway reads the user's role from the validated JWT and applies path- and method-specific rules. This design avoids duplicating the same authorization matrix across multiple microservices.

7.3 PUBLIC AND PROTECTED APIs

The gateway explicitly treats selected product retrieval operations such as product detail, category listing, product search and best-selling products as public APIs. Protected operations require a valid JWT and appropriate role. The payment webhook is also treated as a special integration endpoint and should be protected by provider signature verification at the payment service.

7.4 TRUSTED USER HEADERS

After authorization, the gateway adds X-User-Id, X-User-Email and X-User-Role headers to the forwarded request. Controllers such as ProductController and OrderController consume these headers for vendor/customer identity. These headers must only be trusted when requests reach the downstream service through the controlled gateway path.

8. IMPLEMENTATION DETAILS

8.1 LAYERED SERVICE STRUCTURE

The backend follows a layered structure. REST controllers define HTTP endpoints and validate incoming requests. Service classes implement business logic. Repository/data-access components handle persistence. DTO classes isolate API contracts from internal entities. Utility and infrastructure services provide cross-cutting capabilities such as JWT processing, OTP storage, S3 upload URL generation and notifications.

8.2 DEPENDENCY INJECTION

Spring dependency injection is used to assemble application components. The supplied controllers use Lombok's @RequiredArgsConstructor, which generates constructors for final service dependencies. This keeps controllers focused on request handling and delegates business logic to service classes.

8.3 VALIDATION AND HTTP RESPONSES

Jakarta Validation annotations such as @Valid, @NotNull and @Positive are used for request and identifier validation. ResponseEntity is used to return meaningful HTTP status codes such as 201 Created for successful registration or product creation and 204 No Content for operations that intentionally return no response body.

9. SYSTEM FLOW DIAGRAMS

9.1 LOGIN AND AUTHORIZATION FLOW

Component	Flow
1. Client	User enters credentials and calls /auth/login.
2. User Service	Validates credentials and returns JWT/refresh token information.
3. Client	Stores the access token and sends Authorization: Bearer <token>.
4. API Gateway	Validates JWT and extracts userId, email and role.
5. API Gateway	Checks role against endpoint authorization rules.
6. Target Service	Receives authorized request with X-User-* headers.
7. Response	Target service response is returned through the gateway.

9.2 PRODUCT CREATION FLOW

Step	Description
1	Vendor logs in and obtains a JWT.
2	Vendor sends POST /api/products through the gateway.
3	Gateway validates JWT and verifies ROLE_VENDOR.
4	Gateway adds X-User-Id and X-User-Role.
5	Product Service receives vendor identity and product DTO.
6	Product Service stores the product and returns the created result.
7	Gateway forwards the response to the frontend.

9.3 PAYMENT WEBHOOK FLOW

Step	Description
1	Payment provider sends webhook request.
2	Webhook includes raw body, signature and timestamp.
3	Payment Service verifies the webhook.
4	Verified payment status is processed and persisted.
5	HTTP 200 response acknowledges successful processing.

10. TESTING

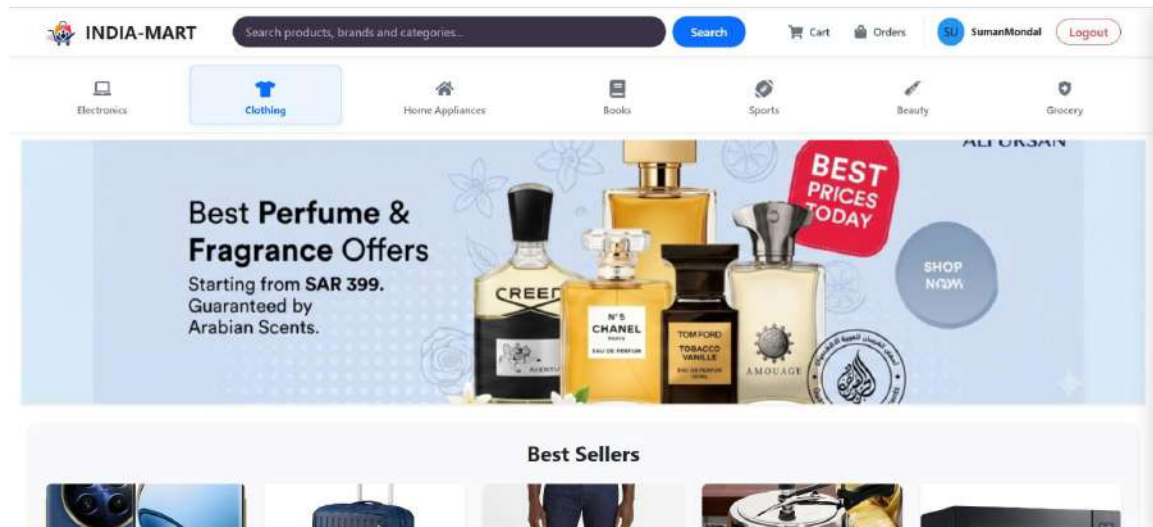
10.1 TEST CASES

SR-NO	TEST CASE	EXPECTED RESULT	ACTUAL RESULT	ERROR MESSAGE
1	Customer signup	Signup initiated and OTP sent.	Pass	None
2	Vendor signup	Vendor OTP flow starts successfully.	Pass	None
3	Invalid OTP	Registration rejected with invalid/expired OTP message.	Pass	None
4	Login	Valid credentials return token response.	Pass	None
5	Invalid login	Authentication request rejected.	Pass	Authentication failure
6	Add product as vendor	Product is created.	Pass	None
7	Add product as customer	Gateway denies request.	Pass	Access denied
8	Edit product as vendor	Product is updated.	Pass	None
9	Delete product as unauthorized role	Gateway denies request.	Pass	Access denied
10	Public product search	Product results returned without protected role check.	Pass	None
11	Customer creates order	Order is created.	Pass	None
12	Vendor updates order status	Vendor order status is updated.	Pass	None
13	Payment webhook	Webhook is verified and processed.	Pass	None
14	Admin views payments	Payment details returned to administrator.	Pass	None

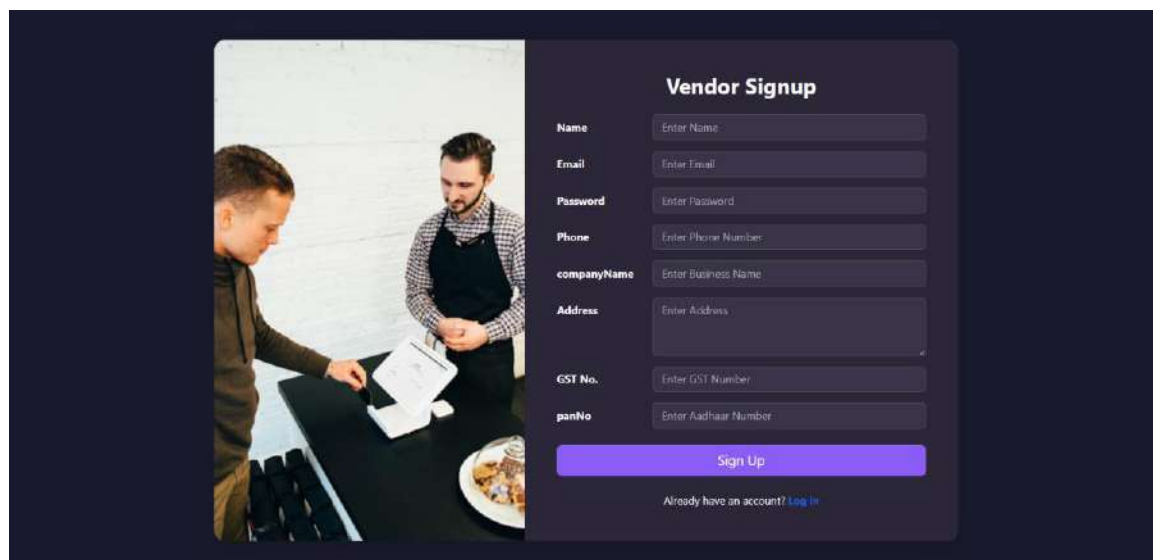
The test cases above represent the intended functional verification matrix based on the supplied controllers and gateway authorization rules. Actual execution evidence, screenshots and exact response payloads can be added to the final submission after the team's final integrated test run.

11. Screenshots

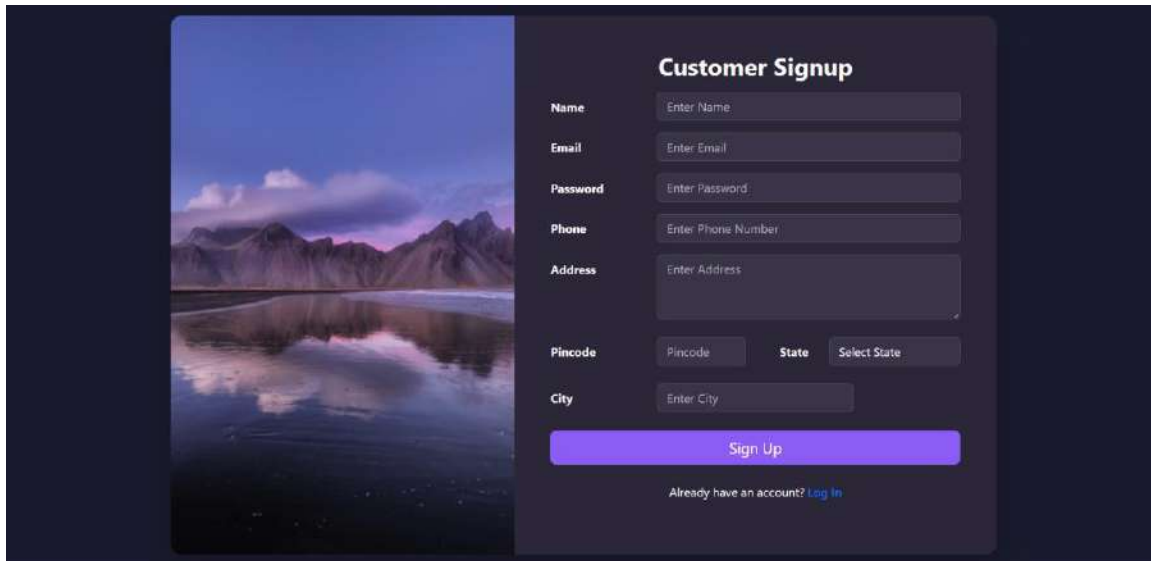
CUSTOMER



Home Page

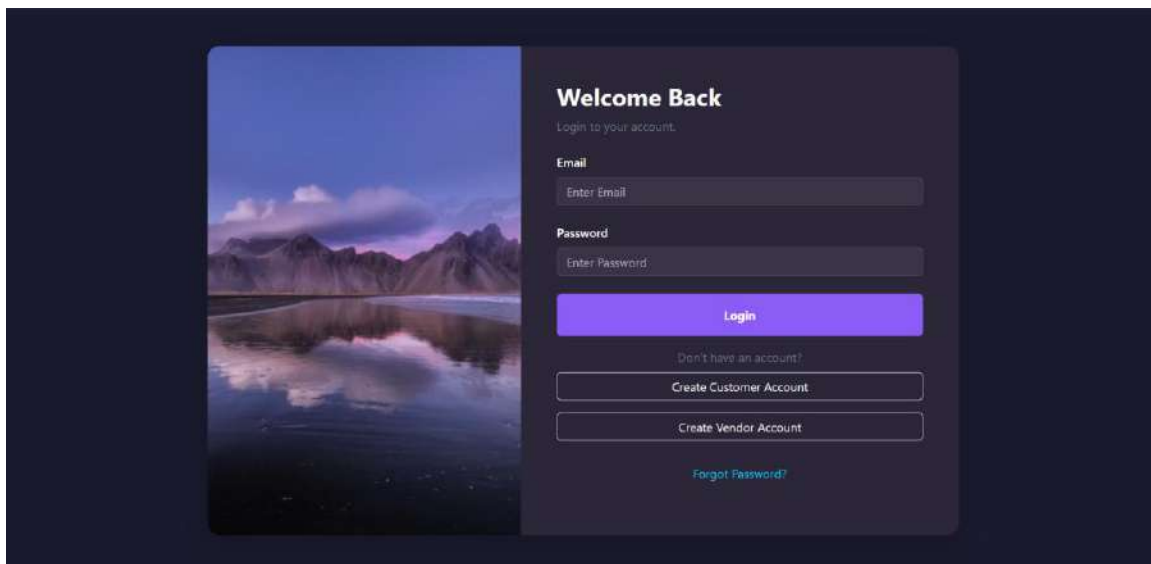


Vendor Signup



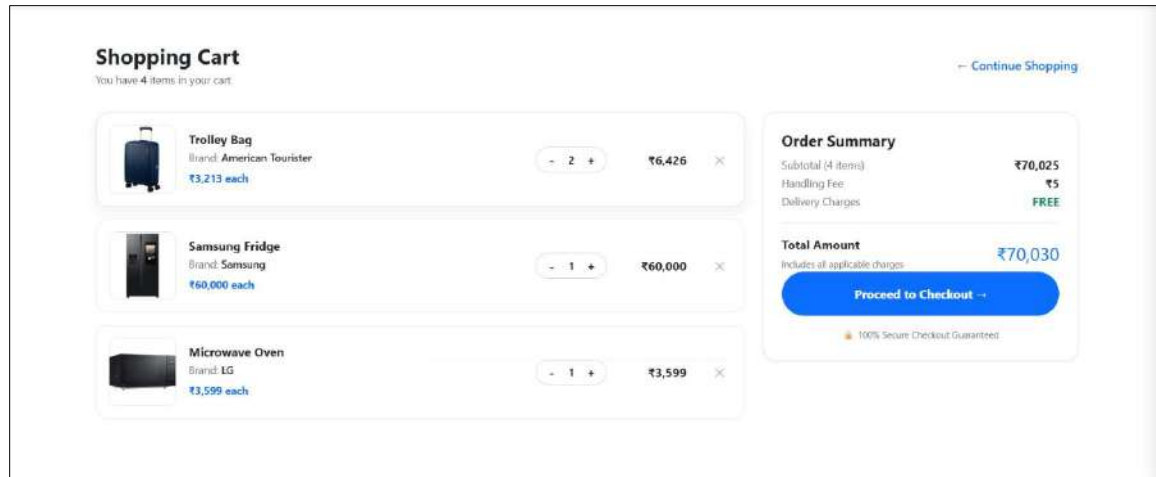
The image shows a 'Customer Signup' form on a dark background. On the left is a vertical rectangular image of a mountain range reflected in water under a purple and blue sky. The form is on the right and contains the following fields: Name (text input), Email (text input), Password (text input), Phone (text input), Address (text input), Pincode (text input), State (dropdown menu labeled 'Select State'), and City (text input). Below these fields is a large orange 'Sign Up' button. At the bottom, there is a link that says 'Already have an account? Log In'.

Customer Signup

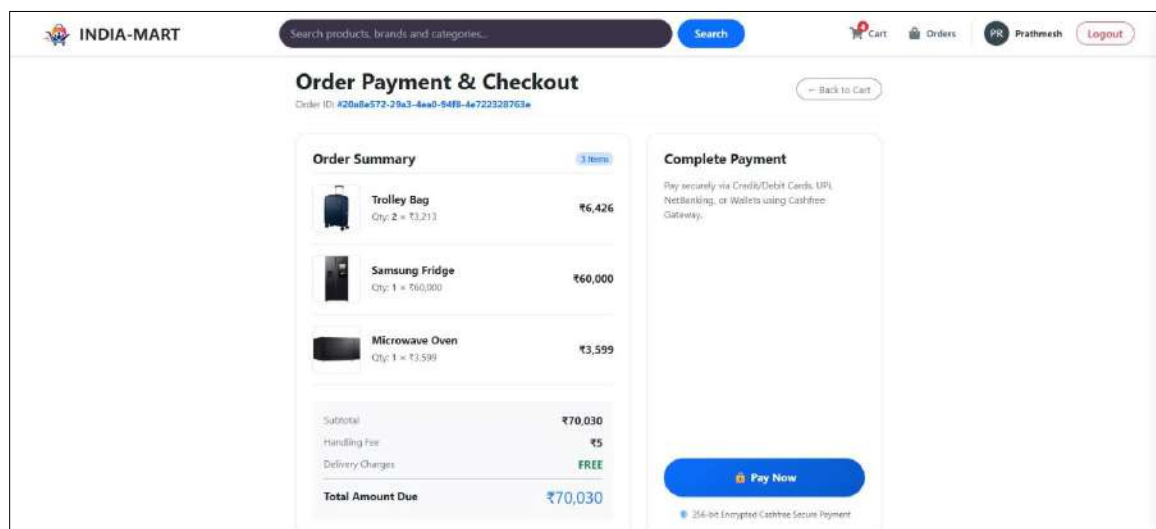


The image shows a 'Welcome Back' login form on a dark background. On the left is a vertical rectangular image of a mountain range reflected in water under a purple and blue sky. The form is on the right and contains the following fields: Email (text input) and Password (text input). Below these fields is a large orange 'Login' button. Under the 'Login' button, there is a link that says 'Don't have an account?'. Below that are two buttons: 'Create Customer Account' and 'Create Vendor Account'. At the bottom, there is a link that says 'Forgot Password?'.

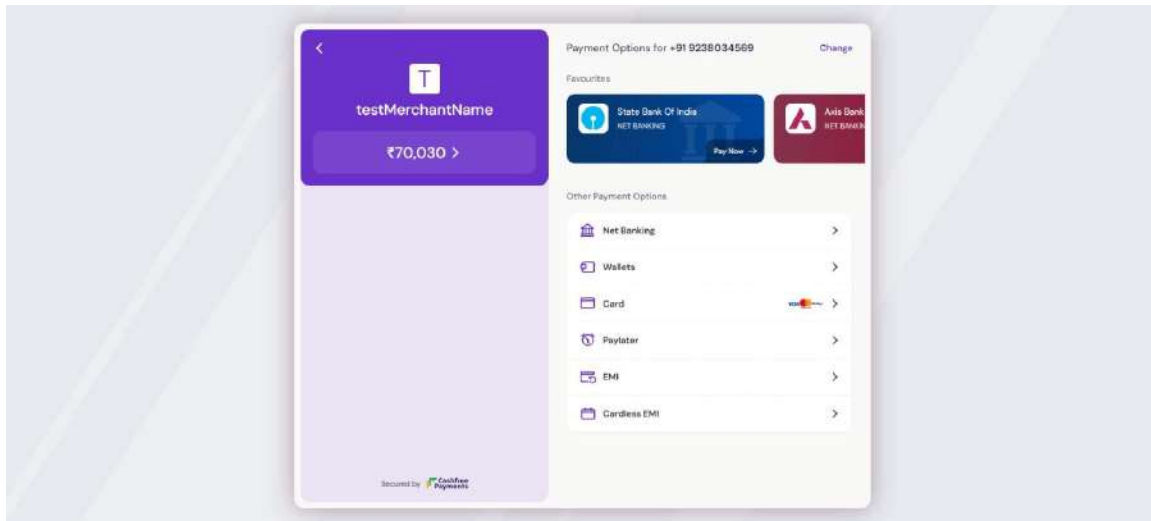
Login for All



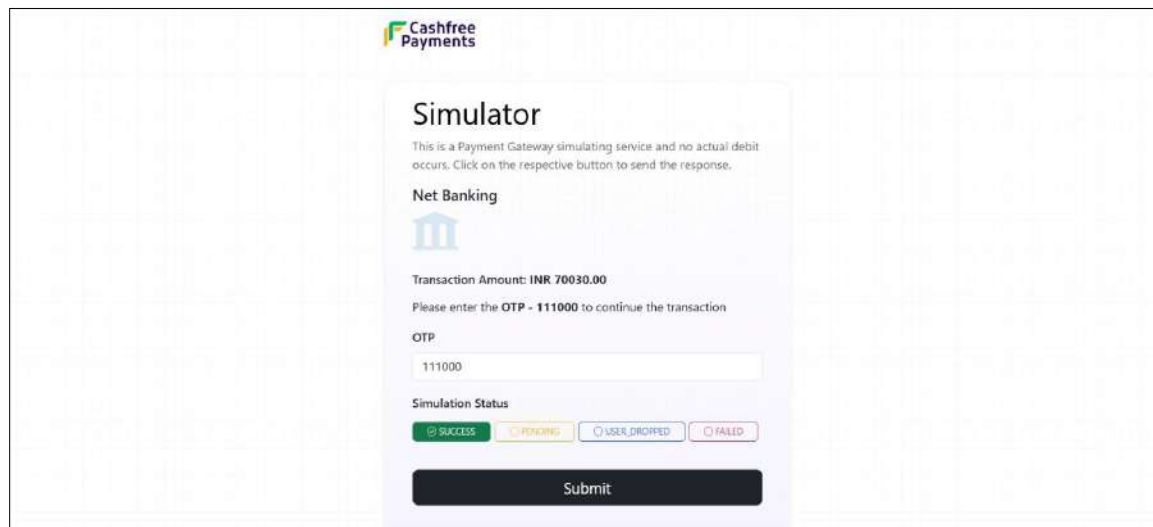
Cart Page



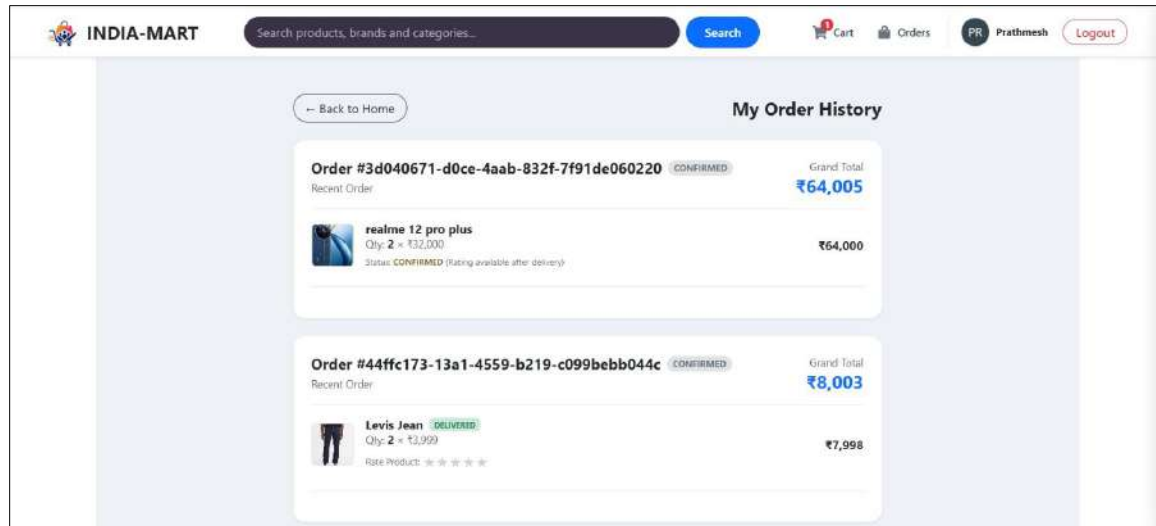
Order Summery Page



Payment Page

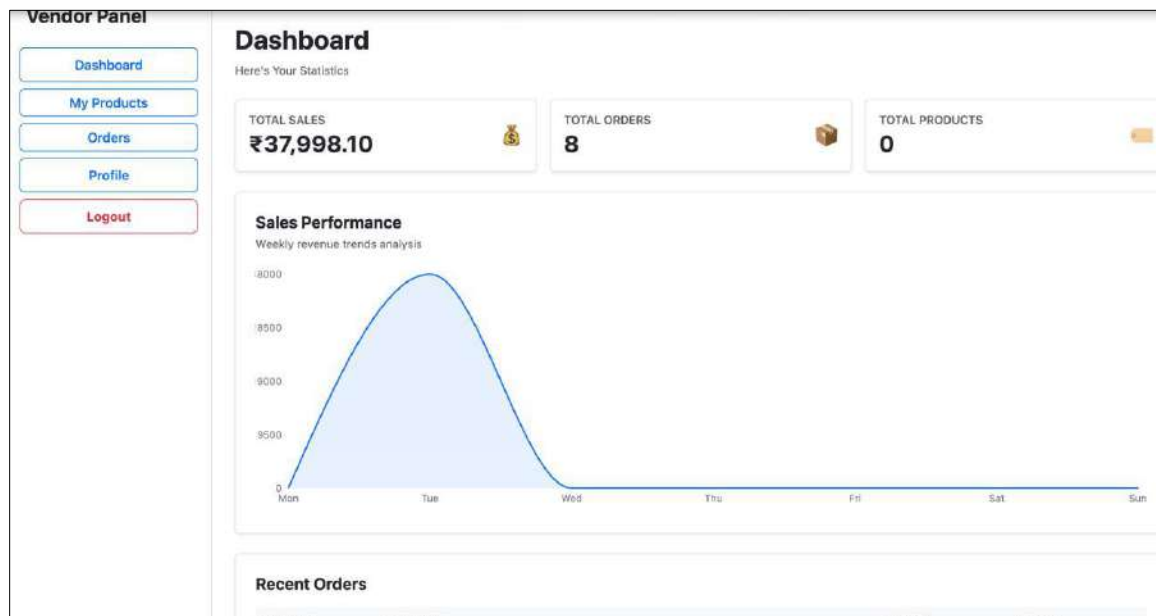


Payment Processing Page



Order History

VENDOR



Vendor Dashboard

Multi-Vendor E-Commerce Marketplace

Vendor Panel

Dashboard

My Products

Orders

Profile

Logout

My Products

Manage store listings and track active inventory levels

Low Stock Alerts

4 items



Study Table

₹2,200.00

Only 2 left



Acer Nitro Gaming Laptop

₹80,000.00

Only 2 left



Being Human Cotton Shirt

₹1,200.00

Only 2 left



Smart Watch

₹52,000.00

Only 2 left

All Products Catalog

Overview of active running store listings



Samsung Fridge

₹90,000.00

891 items



realme 12 pro plus

₹32,000.00

11 items



Study Table

₹2,200.00

2 left



Acer Nitro Gaming Laptop

₹80,000.00

2 left



Being Human Cotton Shirt

₹1,200.00

2 left



Trolley Bag

₹3,213.00

843 items



Smart Watch

₹52,000.00

2 left



Levis Jean

₹3,999.00

42 items

Add Product

Vendor's Products

Dashboard

Incoming Orders

Track customer purchases, fulfillment cycles, and products

ORDER ID	PRODUCTS	SUBTOTAL	STATUS	ACTIONS
ORD-1	realme 12 pro plus (x1)	₹32000	DELIVERED	DELIVERED
ORD-2	Levis Jean (x2)	₹7998	DELIVERED	DELIVERED
ORD-3	Pressure Cooker (x1)	₹1599	OUT FOR DELIVERY	OUT FOR DELIVERY
ORD-4	realme 12 pro plus (x2)	₹64000	PENDING	PENDING
ORD-5	Trolley Bag (x1)	₹3213	PENDING	PENDING
ORD-6	Trolley Bag (x1)	₹3213	PENDING	PENDING
ORD-7	iPhone 17 (x1)	₹433	PENDING	PENDING
ORD-8	Trolley Bag (x2) Samsung Fridge (x1) Microwave Oven (x1)	₹70025	PENDING	PENDING

Vendor's Orders

22

Vendor Panel

Dashboard

My Products

Orders

Profile

Logout

Vendor Profile

PAN: ABDAE17345

Official registration credentials and profile summary

Full Name

Suman Mondal

Phone Number

9123400890

Email Address

sumanmondalmb71@gmail.com

Account Password

Legal Business Name (Company)

pattern

GSTIN / Tax Registration

27ABCD123456789

PAN Number

ABDAE17345

Registered Operational Address

West Bengal

Edit Profile

Vendor Profile

ADMIN

IndiaMart

Dashboard

Customers

Vendors

Products

Orders

Payments

SM Suman Mondal Admin

Logout

Admin Dashboard

Monitor key metrics and manage platform efficiently

\$ Weekly Revenue →

1999.9

📦 Weekly Orders →

8

👤 Customers →

7

🏪 Vendors →

5

Revenue Overview

A line chart titled 'Revenue Overview' showing revenue over a week. The y-axis ranges from 0 to 2000 in increments of 500. The x-axis shows days from Monday to Sunday. The revenue starts at 0 on Monday, rises to a peak of approximately 2000 on Tuesday, and then drops to 0 for the rest of the week (Wednesday through Sunday).

Day	Revenue
Mon	0
Tue	2000
Wed	0
Thu	0
Fri	0
Sat	0
Sun	0

Admin Dashboard

23

IndiaMart

Dashboard

Customers

Vendors

Products

Orders

Payments

SM Suman Mondal Admin

Logout

Customer Management

Manage customer information and account status efficiently

All Customers

ID	Full Name	Email	Phone No.	Joined On	Status	Action
5	Sazid Mughal	testserverm01@gmail.com	9238034567			<button>Unban</button> <button>Delete</button>
7	Prathmesh	prathmeshrayke1@gmail.com	9238034569			<button>Unban</button> <button>Delete</button>
8	Sumana Mondal	sumanmondalgaming@gmail.com	9224034569			<button>Unban</button> <button>Delete</button>
9	Rishu Maddheshiya	rkms6236@gmail.com	9119629523			<button>Unban</button> <button>Delete</button>
24	cc c j C	weyare1624@bora4d.com	1234569521			<button>Unban</button> <button>Delete</button>
27	Rishu	fc9nm@web-library.net	9877777777			<button>Unban</button> <button>Delete</button>
30	asfvfd	hy5ju02.10@minilts.net	9876789098			<button>Unban</button> <button>Delete</button>

Customer Management

IndiaMart

Dashboard

Customers

Vendors

Products

Orders

Payments

SM Suman Mondal Admin

Logout

Product Management

Search products using the product ID.



OnePlus Nord CE 4

Smooth and powerful smartphone with a vibrant display and fast charging.

Product ID: 23

Brand: OnePlus

Category: ELECTRONICS

Price: ₹24999

Available Quantity: 200

Reserved Quantity: 0

Average Rating: 0 ★

Total Ratings: 0

Status: Unpublished

Vendor ID: 6

Created On: 9/8/2026, 3:33:51 pm

Updated On: 9/8/2026, 3:33:51 pm

Admin Product Management

IndiaMart

Dashboard

Customers

Vendors

Products

Orders

Payments

SM Suman Mondal Admin

Logout

Payments

Manage transactions

Recent Payments

Search payment ID

Search

Payment ID	Order ID	Customer	Email	Amount	Method	Status	Created
207317891532064	20a8e572-29a3-4ea0-94f8-4e722328763e	Prathmesh	prathameshrayke1@gmail.com	₹70090	net_banking	SUCCESS	09 Aug 2026
207229302010912	f162537c-15a7-479a-bbd5-6af99c7830c5	Prathmesh	prathameshrayke1@gmail.com	₹488	net_banking	SUCCESS	09 Aug 2026
207225857508640	731ac8bf-26b5-473c-bd6d-b30b90362529	Prathmesh	prathameshrayke1@gmail.com	₹3218	net_banking	SUCCESS	09 Aug 2026
206746574398464	71f0d2de-b8af-41df-b120-ccc13b87c9fa	Prathmesh	prathameshrayke1@gmail.com	₹3218	net_banking	SUCCESS	08 Aug 2026
206082917683584	3d040671-d0ce-4aeb-832f-7f91de060220	Prathmesh	prathameshrayke1@gmail.com	₹64005	net_banking	SUCCESS	06 Aug 2026
1438288056628231168	5e91216d-5ec5-494f-a8df-38d185b3355b	Prathmesh	prathameshrayke1@gmail.com	₹1604	net_banking	SUCCESS	04 Aug 2026
1438283609739049984	44ffc173-13a1-4559-b219-c099bebb044c	Prathmesh	prathameshrayke1@gmail.com	₹8003	net_banking	SUCCESS	04 Aug 2026
1438239406615116800	bf1c4280-5460-4e40-9014-7486fbc1eb86	Prathmesh	prathameshrayke1@gmail.com	₹32005	net_banking	SUCCESS	04 Aug 2026
5114933518777	43663821-3de4-4a38-9227-e3730cd7c45d	Prathmesh	prathameshrayke1@gmail.com	₹4004	net_banking	SUCCESS	03 Aug 2026
5114933514140	846bdb99-898e-426b-bb40-22e6a1801003	Prathmesh	prathameshrayke1@gmail.com	₹480005	net_banking	SUCCESS	03 Aug 2026

Transaction Management

IndiaMart

Dashboard

Customers

Vendors

Products

Orders

Payments

SM Suman Mondal Admin

Logout

Admin Profile

Manage your personal information and security settings

SM

Suman Mondal

Super Admin

CONTACT DETAILS

Email Address

admin@careerace.com

Phone Number

+91 9876543210

Location

Mumbai, India

Personal Information

First Name

Suman

Last Name

Mondal

Email Address

admin@careerace.com

Phone Number

+91 9876543210

Location

Mumbai, India

Save Changes

Change Password

Current Password

New Password

Confirm Password

Update Password

Admin Profile

25

12. DEPLOYMENT AND DEVOPS

The microservices can be packaged and deployed independently. Maven is used to manage dependencies and build Spring Boot services. Docker can be used to containerize the services and infrastructure such as Redis and message brokers. A service-discovery component can be used to register services and allow gateway routing to remain independent of fixed service addresses.

12.1 EXPECTED DEPLOYMENT COMPONENTS

Component	Role
Frontend	Web client for customers, vendors and administrators.
API Gateway	Single external entry point and centralized authorization.
User Service	Identity, authentication and account management.
Product Service	Product domain operations.
Order Service	Order domain operations.
Payment Service	Payment domain and webhook processing.
Service Discovery	Service registration and discovery.
Redis	OTP and temporary signup state.
Message Broker / Notification	Asynchronous notification processing.
MySQL / Databases	Persistent domain data.
Amazon S3	Product images.
Docker / Containers	Packaging and deployment.

13. CHALLENGES AND SOLUTIONS

Challenge	Solution
Duplicating authorization logic in every service	Centralized role/path authorization at the API Gateway.
Secure password storage	BCrypt password hashing.
OTP data should not become permanent user data	Temporary Redis storage until OTP verification.
Large image uploads	S3 pre-signed URLs allow direct object upload.
Different roles require different permissions	JWT role extraction and centralized role-based rules.
Service coupling	REST-based service boundaries and DTO contracts.
Payment callback security	Webhook signature and timestamp verification in Payment Service.

14. FUTURE SCOPE

- Add comprehensive centralized exception handling and standardized error-response contracts across all services.
- Introduce distributed tracing and correlation IDs for easier debugging across microservices.
- Add circuit breakers, retries and timeouts for resilient service-to-service communication.
- Expand automated unit, integration and end-to-end test coverage.
- Introduce CI/CD pipelines for automated build, test and deployment.
- Add API documentation using OpenAPI/Swagger for every service.
- Add monitoring and metrics using suitable observability tools.
- Complete and document the remaining microservices in the same level of detail as the services already supplied.

15. CONCLUSION

The Multi-Vendor E-Commerce Marketplace demonstrates how a traditional e-commerce application can be decomposed into independently manageable microservices. The architecture separates identity, product, order and payment responsibilities while using an API Gateway as the controlled entry point for client requests.

The most important security responsibility is divided between authentication and authorization. The User Service authenticates users and protects passwords with BCrypt, while the API Gateway validates JWTs and centrally enforces role-based authorization. The gateway then forwards trusted user information to downstream services, allowing those services to concentrate on business rules and ownership validation.

The submitted implementation also demonstrates practical distributed-system features such as Redis-based OTP storage, asynchronous notification, S3 pre-signed image uploads, payment webhooks and role-specific order and product operations. With the remaining service source files and final integration-test evidence, this report can be extended into a complete institutional project submission.

16. REFERENCES

- [1] Spring Boot, “Spring Boot Reference Documentation,” VMware, Inc. [Online]. Available: <https://docs.spring.io/spring-boot/index.html>. [Accessed: Aug. 10, 2026].
- [2] Spring, “Spring Cloud Reference Documentation,” VMware, Inc. [Online]. Available: <https://docs.spring.io/spring-cloud/docs/current/reference/html/>. [Accessed: Aug. 10, 2026].
- [3] Meta Platforms, Inc., “React Documentation.” [Online]. Available: <https://react.dev/>. [Accessed: Aug. 10, 2026].
- [4] Oracle Corporation, “MySQL 8.4 Reference Manual.” [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/>. [Accessed: Aug. 10, 2026].
- [5] RabbitMQ, “RabbitMQ Documentation.” [Online]. Available: <https://www.rabbitmq.com/docs>. [Accessed: Aug. 10, 2026].
- [6] Docker Inc., “Docker Documentation.” [Online]. Available: <https://docs.docker.com/>. [Accessed: Aug. 10, 2026].
- [7] Redis Ltd., “Redis Documentation.” [Online]. Available: <https://redis.io/docs/latest/>. [Accessed: Aug. 10, 2026].
- [8] Amazon Web Services, “Amazon S3 Documentation.” [Online]. Available: <https://docs.aws.amazon.com/s3/>. [Accessed: Aug. 10, 2026].
- [9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>.
- [10] Cashfree Payments, “Cashfree Payments Developer Documentation.” [Online]. Available: <https://www.cashfree.com/docs/>. [Accessed: Aug. 10, 2026].
- [11] Spring, “Spring Security Reference Documentation,” VMware, Inc. [Online]. Available: <https://docs.spring.io/spring-security/reference/>. [Accessed: Aug. 10, 2026].
- [12] Spring, “Spring Data JPA Reference Documentation,” VMware, Inc. [Online]. Available: <https://docs.spring.io/spring-data/jpa/reference/>. [Accessed: Aug. 10, 2026].
- [13] Mozilla Developer Network, “HTTP — MDN Web Docs.” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP>. [Accessed: Aug. 10, 2026].